ĐỒ ÁN CÔNG NGHỆ PHẦN MỀM - THIẾT KẾ KIẾN TRÚC NÂNG CAO

HƯỚNG DẪN ÁP DỤNG CACHING TOÀN DIỆN CHO HỆ THỐNG SOOBIN SWIMWEAR

Chiến lược Caching Đa Tầng, Tối ưu Hiệu năng Truy vấn, Đảm bảo Tính Nhất quán Dữ liệu (ACID) & Thỏa mãn Tiêu chí Nghiệm thu PRD v0.2

DỰ ÁN THAM CHIẾU
Website Thương mại Điện tử Đồ bơi Soobin Swimwear (CNPMidea)

TÀI LIỆU NỀN TẢNG
mo_ta_cnpm.docx.pdf & prd.docx.pdf (v0.2)

NGĂN XẾP CÔNG NGHỆ
FastAPI (Python) + PostgreSQL + React (Vite) + Docker Compose

MÔ HÌNH KIẾN TRÚC BẮT BUỘC
3-layer Architecture (Controller/API → Service → Repository)

NHÂN SỰ & THỜI HẠN
5 thành viên (2 FE, 3 BE) | Hoàn thành MVP v1: 25/11/2026

CHUYÊN ĐỀ KỸ THUẬT
Multi-Tier Caching, Invalidation Patterns & ACID Inventory Protection

Lưu hành nội bộ nhóm phát triển CNPM

Phiên bản 1.0 — Tháng 10/2026

MỤC LỤC & TỔNG QUAN ĐỀ ÁN

CHƯƠNG 1: BỐI CẢNH DỰ ÁN & ĐÁNH GIÁ NHU CẦU CACHING ..... Trang 3
- 1.1 Tổng quan dự án từ mo_ta_cnpm.docx.pdf và prd.docx.pdf (v0.2)
- 1.2 Phân tích đặc thù lưu lượng: Read-Heavy vs Write-Sensitive
- 1.3 Mục tiêu hiệu năng (NFR-07) và các tiêu chí nghiệm thu liên quan
- 1.4 Ranh giới an toàn: Dữ liệu ĐƯỢC PHÉP CACHE vs TUYỆT ĐỐI KHÔNG ĐƯỢC CACHE

CHƯƠNG 2: MÔ HÌNH KIẾN TRÚC CACHING ĐA TẦNG (MULTI-TIER CACHING) ..... Trang 5
- 2.1 Sơ đồ luồng dữ liệu 4 tầng (Client → Proxy → App Server → Database)
- 2.2 Tầng 1: Client-Side Caching (TanStack React Query & LocalStorage)
- 2.3 Tầng 2: Reverse Proxy & Static Asset Caching (Nginx / Edge)
- 2.4 Tầng 3: Application Server Caching (FastAPI In-Memory & Redis)
- 2.5 Tầng 4: Database Buffer Pool & Query Optimization trong PostgreSQL

CHƯƠNG 3: HƯỚNG DẪN CACHING CHI TIẾT THEO TỪNG PHÂN HỆ NGHIỆP VỤ ..... Trang 7
- 3.1 Phân hệ Catalog & Storefront (Danh mục, kích thước, màu sắc, chi tiết SP)
- 3.2 Phân hệ Khuyến mãi & Voucher (Bảng CTKM, công thức 3 phương án, quota)
- 3.3 Phân hệ Xác thực & Phân quyền (Auth & RBAC - Thỏa mãn AC-AUTH-02 tức thì)
- 3.4 Phân hệ Giỏ hàng & Tồn kho (Cart & Inventory - Bảo vệ nghiêm ngặt AC-CHK-01)
- 3.5 Phân hệ Báo cáo & Dashboard Quản trị (Pre-computed Summary Caching)

CHƯƠNG 4: CHIẾN LƯỢC ĐỒNG BỘ & VÔ HIỆU HÓA CACHE (CACHE INVALIDATION) ..... Trang 9
- 4.1 Mô hình Đọc/Ghi: Cache-Aside (Lazy Loading) + Write-Invalidate
- 4.2 Thiết lập TTL Phân Tầng (Tiered Expiration Strategy)
- 4.3 Phòng chống 3 sự cố: Cache Penetration, Avalanche và Stampede
- 4.4 Cơ chế Tự động Vô hiệu hóa qua Middleware & Service Hooks

CHƯƠNG 5: THIẾT KẾ KỸ THUẬT & MÃ NGUỒN MINH HỌA ..... Trang 10
- 5.1 Thiết kế CacheManager tuân thủ chuẩn 3 tầng (Controller/API → Service → Repository)
- 5.2 Mã nguồn module app/core/cache.py (In-Memory & Redis Adapter)
- 5.3 Tích hợp thực tế vào CatalogService và AuthService
- 5.4 Cấu hình bổ sung dịch vụ Redis trong docker-compose.yml

CHƯƠNG 6: LỘ TRÌNH TRIỂN KHAI PHÙ HỢP DEADLINE 25/11/2026 ..... Trang 14
- 6.1 Lộ trình 3 giai đoạn: MVP Baseline → Docker Demo → Production Scale
- 6.2 Phân công nhiệm vụ chi tiết cho 2 Frontend & 3 Backend

CHƯƠNG 7: MA TRẬN CACHING TỔNG HỢP (MASTER CACHING MATRIX) ..... Trang 15

CHƯƠNG 8: KỊCH BẢN KIỂM THỬ & NGHIỆM THU CACHING (TEST CASES) ..... Trang 16

> **Tóm lược định hướng kiến trúc:**
> Tài liệu này giải quyết triệt để bài toán hiệu năng (NFR-07: tải trang dưới 3 giây cho 500 biến thể) thông qua giải pháp Caching thông minh, đồng thời bảo vệ 100% tính toàn vẹn của dữ liệu giao dịch tài chính và kho hàng (AC-CHK-01: không âm kho; AC-AUTH-02: khóa tài khoản tức thì).

CHƯƠNG 1: BỐI CẢNH DỰ ÁN & ĐÁNH GIÁ NHU CẦU CACHING

1.1 Tổng quan dự án từ 2 tài liệu nền tảng

Dự án phát triển nền tảng thương mại điện tử chuyên biệt cho ngành hàng đồ bơi (**Soobin Swimwear**) được định hình bởi 2 tài liệu cốt lõi:

- `mo_ta_cnpm.docx.pdf`: Đặc tả tổng thể các phân hệ nghiệp vụ gồm Quản lý phân quyền người dùng, Quản lý danh mục & sản phẩm đồ bơi (theo size, màu, bảng số đo chi tiết), Quản lý giỏ hàng, Thanh toán COD/Chuyển khoản VietQR, Quản lý phiếu nhập kho ghi nhận lịch sử và tính giá vốn, Quản lý đơn hàng/hóa đơn và Quản lý khuyến mãi.
- `prd.docx.pdf` **(v0.2)**: Quy định chi tiết các quyết định kỹ thuật đã chốt: Ngăn xếp công nghệ **FastAPI (Python) + PostgreSQL + React (Vite)**; Kiến trúc bắt buộc **3-layer (Controller/API → Service → Repository)**; Đóng gói và chạy demo bằng **Docker Compose**; Hạn chót hoàn thành MVP v1 chính xác vào ngày **25/11/2026** bởi nhóm 5 thành viên (2 FE, 3 BE).

1.2 Phân tích đặc thù lưu lượng: Read-Heavy kết hợp Write-Sensitive

Trong thương mại điện tử đồ bơi, cấu trúc sản phẩm phức tạp (1 sản phẩm × nhiều màu × nhiều size = hàng trăm biến thể với hình ảnh và số lượng tồn riêng). Lưu lượng truy cập phân hóa rõ rệt:

PHÂN HỆ / HÀNH VI	TỈ TRỌNG	ĐẶC TÍNH I/O	TÁC ĐỘNG & ĐIỂM NGHẼN NẾU KHÔNG CÓ CACHE
Duyệt Catalog & Storefront (Trang chủ, Mega menu, Lọc size/màu, Chi tiết, Outlet)	**85% - 90%**	Read-Heavy Dữ liệu tĩnh/ít biến động	Truy vấn JOIN nhiều bảng (Product × Variant × Inventory × Image), tiêu tốn CPU DB, response time vượt ngưỡng 3s khi duyệt 500 biến thể.
Xác thực & Kiểm tra Quyền (Session token, RBAC Permission)	**100%** private APIs	Read-Heavy Đọc liên tục mỗi request	Mỗi request người dùng đều phải truy vấn các bảng `user_session`, `user_account`, `user_role_permission`.
Tra cứu Khuyến mãi (Active promotions, check voucher)	**30% - 40%**	Read-Heavy Đọc điều kiện CTKM	Tính toán lại công thức 3 phương án liên tục, quét bảng CTKM lặp đi lặp lại trên mỗi giỏ hàng.
Tạo đơn hàng & Trừ kho (Checkout, Khóa tồn kho, Trừ voucher)	**5% - 10%**	Write-Sensitive Giao dịch ACID khắt khe	Cạnh tranh mua hàng cuối (Flash sale). Bắt buộc dùng Khóa bi-quan (Pessimistic Lock) trong DB, **CẤM CACHE LÀM BỎ QUA TRANSACTION**.
Nhập hàng & Giá vốn (Xác nhận phiếu nhập, bình quân)	Nội bộ	Write-Sensitive Tính toán giá vốn hàng bán	Cập nhật trực tiếp giá vốn bình quân gia quyền phục vụ snapshot giá vốn của đơn hàng.

1.3 Mục tiêu hiệu năng & Ràng buộc theo PRD

- **NFR-07 (PRD mục 13):** Thời gian tải trang danh mục và chi tiết sản phẩm **dưới 3 giây** với quy mô khoảng 500 biến thể.
- **AC-AUTH-02 (PRD mục 4):** Đổi permission hoặc khóa tài khoản phải **có hiệu lực ngay với yêu cầu tiếp theo**, không cần chờ phiên hết hạn.
- **AC-CHK-01 (PRD mục 5):** Hai khách hàng đồng thời mua sản phẩm/biến thể cuối cùng **chỉ tối đa một đơn tạo thành công**, tồn kho không âm.
- **BR-PROMO-01-11 (PRD mục 7):** Voucher có giới hạn lượt dùng, chỉ trừ lượt khi phương án voucher thắng và tạo đơn thành công; không trừ lặp.

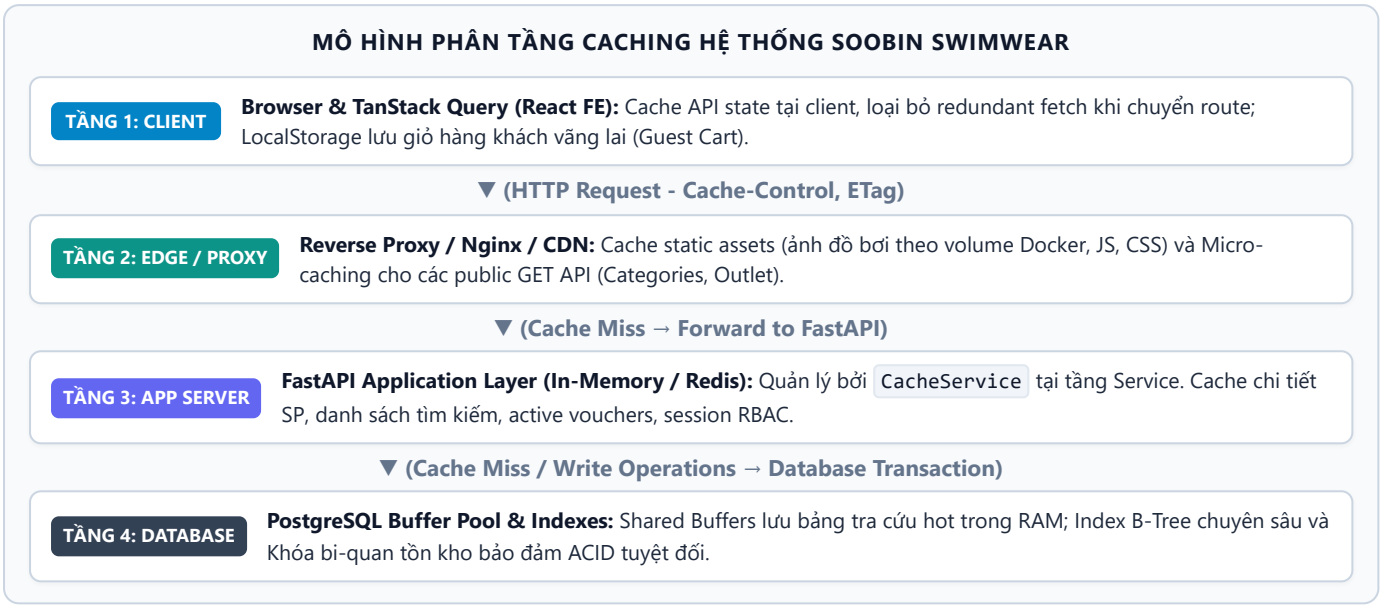
1.4 Ranh giới an toàn: Dữ liệu ĐƯỢC CACHE vs TUYỆT ĐỐI KHÔNG ĐƯỢC CACHE

DỮ LIỆU ĐƯỢC PHÉP CACHE (AN TOÀN - NÊN CACHE)	DỮ LIỆU TUYỆT ĐỐI KHÔNG ĐƯỢC CACHE (NGUY HIỂM - CẤM CACHE)
✓ Danh mục sản phẩm (Categories), Kích thước (Sizes), Màu sắc (Colors). ✓ Chi tiết sản phẩm, bảng thông số kỹ thuật, bảng số đo, hình ảnh. ✓ Kết quả lọc/tìm kiếm sản phẩm trên Storefront. ✓ Danh sách chương trình khuyến mãi đang kích hoạt. ✓ Cấu hình hệ thống (Site Settings, Banner, Thông báo chạy). ✓ Thông tin Session người dùng (kèm cơ chế Invalidate tức thì khi bị khóa). ✓ Báo cáo Dashboard tổng hợp theo ngày/tháng (Aggregate metrics).	✕ **Số lượng tồn kho khả dụng khi Checkout:** Phải đọc trực tiếp từ PostgreSQL kèm `SELECT ... FOR UPDATE`. ✕ **Lượt sử dụng Voucher khi tạo đơn:** Phải kiểm tra và tăng `used_count` trong transaction CSDL. ✕ **Số dư tài chính, Số tiền thanh toán đơn:** Phải chốt snapshot cố định từ Service. ✕ **Trạng thái thanh toán VietQR / Webhook vận chuyển:** Phải ghi nhận trực tiếp vào bảng giao dịch bền vững. ✕ **Phiếu nhập hàng đã xác nhận:** Không can thiệp cache làm sai lệch công thức giá vốn bình quân gia quyền.

CHƯƠNG 2: MÔ HÌNH KIẾN TRÚC CACHING ĐA TẦNG

2.1 Sơ đồ luồng dữ liệu 4 tầng (Multi-Tier Caching)

Hệ thống áp dụng mô hình phân tầng chặt chẽ, tối ưu từ Client đến Database mà vẫn bảo toàn tính toàn vẹn 3 tầng của Backend:



2.2 Tầng 1: Client-Side Caching (TanStack React Query & LocalStorage)

- **TanStack Query (React FE):**
  - Dữ liệu Master (Danh mục, kích thước, màu sắc, site settings): Thiết lập `staleTime: 1000 * 60 * 30` (30 phút), `gcTime: 1000 * 60 * 60` (1 giờ).
  - Danh sách sản phẩm Storefront / Chi tiết sản phẩm: `staleTime: 1000 * 60 * 3` (3 phút). Khi người dùng nhấn nút Back/Forward, dữ liệu hiển thị tức thì không cần tải lại.
  - Giỏ hàng tài khoản: `staleTime: 0` (luôn fetch mới khi component mount) nhưng kết hợp Optimistic UI khi tăng/giảm số lượng.
- **Lưu trữ giỏ hàng khách vãng lai (Guest Cart):** Lưu trực tiếp tại `localStorage` (chuẩn hóa theo `BR-PRD-CART-01`), chỉ đồng bộ gộp giỏ (Merge Cart) khi người dùng tiến hành đăng nhập tại bước checkout.
- **HTTP Caching Headers:** Backend FastAPI cấu hình Header `Cache-Control: public, max-age=86400, stale-while-revalidate=3600` đối với thư mục tĩnh `/static` (ảnh sản phẩm, banner).

2.3 Tầng 2: Reverse Proxy & Static Asset Caching

- **Caching hình ảnh áo tắm:** File ảnh lưu trong Docker Volume. Nginx đóng vai trò Reverse Proxy lưu đệm các file ảnh đã truy cập, phản hồi mã HTTP `304 Not Modified` qua cơ chế `ETag`, giảm 95% băng thông gửi về backend FastAPI.
- **Micro-Caching (1 - 5 giây):** Áp dụng cho các endpoint công khai có lượng truy cập đột biến trong thời gian Flash Sale (ví dụ: `/api/v1/catalog/outlet`), giảm thiểu tình trạng hàng ngàn client cùng gửi request tới backend trong cùng 1 giây (Cache Stampede).

2.4 Tầng 3: Application Server Caching (Trọng tâm FastAPI 3-layer)

> **Vị trí đặt Caching trong Kiến trúc 3 Tầng:**
> Theo mục 14 PRD: "*Controller không gọi Repository trực tiếp và không đặt công thức nghiệp vụ... Service điều phối transaction và use-case... Repository chỉ đọc/ghi PostgreSQL*". Do đó:
> **Logic Caching PHẢI NẰM TẠI TẦNG SERVICE**
> (thông qua `CacheService`) hoặc tại
> **REPOSITORY CACHED-READ**
> dưới sự điều khiển vô hiệu hóa của Service. Controller hoàn toàn độc lập với cache!

- **Pha 1 (In-Memory Cache):** Sử dụng `cachetools.TTLCache` hoặc từ điển Python Thread-Safe chạy trực tiếp trong tiến trình FastAPI. Tốc độ siêu tốc (< 0.1ms), zero-dependency, sẵn sàng ngay cho mốc demo MVP v1 (25/11/2026).
- **Pha 2 (Distributed Cache - Redis):** Bổ sung container `redis:7-alpine` vào `docker-compose.yml`. Cho phép chia sẻ cache giữa nhiều workers, lưu trữ session tập trung, hỗ trợ khóa phân tán và xử lý danh sách đen tài khoản bị khóa.

2.5 Tầng 4: Database Buffer Pool & Query Optimization trong PostgreSQL

- **Cấu hình tham số:** Thiết lập `shared_buffers = 256MB` và `effective_cache_size = 512MB` để PostgreSQL ưu tiên giữ các bảng tra cứu hot hoàn toàn trong RAM.
- **Index che phủ (Covering Indexes):** Đảm bảo các truy vấn đọc không phải scan đĩa (Disk I/O) mà được phục vụ 100% từ Index Cache:

```
CREATE INDEX IF NOT EXISTS idx_variant_product_active ON production_variant(product_id, is_active);
CREATE INDEX IF NOT EXISTS idx_product_status_search ON production_product(status, deleted_at, category_id);
CREATE INDEX IF NOT EXISTS idx_inventory_variant_avail ON production_inventory(variant_id, available_quantity);
```

CHƯƠNG 3: HƯỚNG DẪN CACHING CHI TIẾT THEO TỪNG PHÂN HỆ

3.1 Phân hệ Catalog & Storefront (Sản phẩm, Danh mục, Biến thể)

ĐỐI TƯỢNG	ĐỊNH DẠNG KHÓA CACHE (KEY)	TTL	CƠ CHẾ VÔ HIỆU HÓA (INVALIDATION)
Danh mục, Size, Màu (Categories, Sizes, Colors)	`cat:categories` `cat:sizes` `cat:colors`	**24 giờ**	Xóa cache khi Admin thêm/sửa/xóa danh mục hoặc thuộc tính (Rất hiếm khi thay đổi).
Chi tiết sản phẩm (Product Detail, Variants, Images)	`cat:product:{id_or_slug}` Ví dụ: `cat:product:ao-tam-bikini-01`	**30 phút**	Xóa khi Quản lý gọi API cập nhật SP (`PUT /products/{id}`), xóa mềm (`DELETE`) hoặc khi có trừ kho đơn hàng.
Bộ lọc Tìm kiếm (Search & Filter queries)	`cat:search:{hash_of_params}` (MD5 chuỗi sorted query params)	**5 phút**	Tự động hết hạn sau 5 phút, hoặc xóa toàn bộ prefix `cat:search:*` khi có cập nhật sản phẩm mới.
Trang Outlet (Đang giảm giá)	`cat:outlet:list`	**10 phút**	Xóa khi có chương trình khuyến mãi sản phẩm mới được bật/tắt trong `PromotionService`.

3.2 Phân hệ Khuyến mãi & Voucher (Promotions & Vouchers)

- **Cache danh sách CTKM đang hiệu lực:** Khóa `promo:active_list` (TTL: 15 phút). Lưu toàn bộ CTKM thỏa mãn `is_active = True` và thời gian hiện tại nằm trong `starts_at ... ends_at`. Hàm `PromotionService.calculate_discounts()` lấy dữ liệu này từ cache thay vì query bảng `production_promotion` cho mỗi món hàng trong giỏ.
- **Cache tra cứu mã Voucher:** Khóa `voucher:info:{CODE}` (TTL: 5 phút). Lưu thông tin tỷ lệ giảm, ngưỡng tối thiểu, hạn mức tối đa.
- **BẢO VỆ HẠN MỨC SỬ DỤNG VOUCHER (Usage Limit):** Khi xem trước giỏ hàng (Preview), số lượt còn lại đọc từ cache. Khi tạo đơn hàng (`POST /orders`), Service **BẮT BUỘC** kiểm tra và tăng `used_count` trong transaction CSDL: `UPDATE production_promotion SET used_count = used_count + 1 WHERE id = :id AND used_count < usage_limit`. Commit thành công lập tức xóa khóa `voucher:info:{CODE}`.

3.3 Phân hệ Xác thực & Phân quyền (Auth & RBAC - Nghiệm thu AC-AUTH-02)

Tiêu chuẩn **AC-AUTH-02**: *"Đổi permission / khóa tài khoản có hiệu lực với yêu cầu tiếp theo, không cần chờ phiên hết hạn."*

QUY TRÌNH KIỂM TRA QUYỀN & XỬ LÝ KHÓA TÀI KHOẢN TỨC THÌ (AC-AUTH-02)

REQUEST — Client gửi kèm Session Token trong Cookie / Header `Authorization: Bearer ...`

▼

CACHE LOOKUP — `AuthService.validate_session()` kiểm tra cache `auth:session:{token}`. Nếu HIT trả về ngay User ID; nếu MISS đọc PostgreSQL nạp cache (TTL = 30 phút).

▼

ADMIN KHÓA / ĐỔI QUYỀN — Khi Admin gọi API khóa user hoặc sửa permissions: Cập nhật PostgreSQL → **SERVICE XÓA NGAY CACHE** `auth:session:{token}` → Request tiếp theo của User bị từ chối **401/403** lập tức!

3.4 Phân hệ Giỏ hàng & Tồn kho (Cart & Inventory - Bảo vệ AC-CHK-01)

- **Hiển thị Storefront:** Đọc từ cache biến thể với trường `stock_status` : `IN_STOCK` , `LOW_STOCK` , hoặc `OUT_OF_STOCK` (TTL: 60s).
- **Ranh giới Checkout:** Tuyệt đối **CẤM DÙNG DỮ LIỆU TỒN KHO TỪ CACHE ĐỂ ĐẶT HÀNG**. Khi khách bấm "Đặt hàng", `OrderService.create_order()` bắt buộc dùng khóa bi-quan:

```
# Ranh giới an toàn tuyệt đối trong OrderService (đã có trong app/services/order_service.py):
inv = self.inv_repo.get_inventory_for_update(variant_id) # SELECT ... FOR UPDATE
if not inv or inv.available_quantity < qty:
    raise ValueError("Sản phẩm không đủ số lượng tồn kho.")
inv.available_quantity -= qty
# Commit transaction trong PostgreSQL -> Xóa cache chi tiết SP: cache.delete(f"cat:product:{pid}")
```

Thỏa mãn 100% kịch bản kiểm thử `E2E-06` và tiêu chí `AC-CHK-01` ("Hai khách mua món cuối cùng không tạo hai đơn hợp lệ; tồn không âm").

3.5 Phân hệ Báo cáo & Dashboard Quản trị (Dashboard & Reports)

- Truy vấn thống kê doanh thu, lợi nhuận, top bán chạy đòi hỏi `SUM`, `COUNT`, `GROUP BY` trên hàng chục ngàn dòng.
- Áp dụng **Pre-computed Summary Cache** với khóa `dashboard:overview_stats` (TTL: 15 - 30 phút). Đánh dấu hết hạn (Invalidate) khi có đơn hàng chuyển sang trạng thái `Đã giao` (xuất hóa đơn) hoặc `Đã hoàn tiền`.

CHƯƠNG 4: CHIẾN LƯỢC ĐỒNG BỘ & VÔ HIỆU HÓA CACHE

4.1 Mô hình Đọc/Ghi: Cache-Aside (Lazy Loading) + Write-Invalidate

1. **Luồng ĐỌC (Cache-Aside):** Service kiểm tra Cache với key tương ứng. Nếu HIT trả về ngay. Nếu MISS gọi Repository truy vấn PostgreSQL, nạp kết quả vào Cache kèm TTL rồi trả về kết quả.
2. **Luồng GHI (Write-Invalidate):** Service thực thi ghi và **Commit thành công vào PostgreSQL trước**. Sau đó thực hiện **XÓA (Delete/Evict)** key cache tương ứng, KHÔNG cố gắng ghi đè dữ liệu mới vào cache (tránh tranh chấp giữa 2 thao tác ghi đồng thời). Lần đọc sau sẽ nạp dữ liệu chuẩn mới nhất từ CSDL.

4.2 Thiết lập TTL Phân Tầng (Tiered Expiration Strategy)

PHÂN TẦNG TTL	THỜI GIAN KHUYẾN NGHỊ	CÁC ĐỐI TƯỢNG ÁP DỤNG	MỤC TIÊU CHÍNH
Rất Ngắn (Ultra-Short)	**30 giây - 60 giây**	Trạng thái tồn kho hiển thị Storefront, bộ đếm Flash Sale, Micro-cache danh sách Outlet.	Bảo vệ DB trước bão truy vấn trong đợt khuyến mãi.
Ngắn (Short)	**3 phút - 5 phút**	Kết quả tìm kiếm sản phẩm theo từ khóa/bộ lọc, thông tin mã voucher, danh sách đánh giá mới.	Cân bằng giữa độ tươi mới và giảm tải DB.
Trung bình (Medium)	**15 phút - 30 phút**	Chi tiết sản phẩm, danh sách sản phẩm liên quan, thông tin phiên làm việc (Session RBAC), Dashboard.	Tối ưu thời gian duyệt web liên tục của khách hàng.
Dài (Long)	**12 giờ - 24 giờ**	Danh mục cha/con, Bảng kích cỡ (XS, S, M, L), Bảng màu sắc, Cấu hình trang (Banner, Footer, Hotline/Zalo).	Dữ liệu nền tảng gần như không bao giờ đổi trong phiên.

4.3 Phòng chống 3 sự cố Caching kinh điển

- **Cache Penetration (Xuyên thủng cache):** Kẻ xấu truy vấn liên tục các ID không tồn tại.
  Giải pháp: Áp dụng **Null Object Caching**: Khi DB trả về `None`, lưu `{"__null__": True}` vào cache với TTL ngắn (60 giây) để chặn đứng các request lặp lại.
- **Cache Avalanche (Đổ vỡ tuyết lở):** Hàng loạt key đồng loạt hết hạn cùng thời điểm làm sập DB.
  Giải pháp: Áp dụng **TTL Jitter (Độ lệch ngẫu nhiên)**: `actual_ttl = base_ttl + random.randint(-60, 60)`.
- **Cache Stampede (Bầy thú giẫm đạp):** Hàng trăm request cùng lao vào DB khi 1 sản phẩm hot vừa hết hạn cache.
  Giải pháp: Áp dụng **Mutex Lock / Single-Flight** tại Service: Chỉ 1 request đầu tiên được phép query DB và nạp lại cache, các request sau đợi trong 50ms và lấy dữ liệu từ cache.

4.4 Cơ chế Tự động Vô hiệu hóa Cache qua Middleware

Tận dụng middleware sẵn có tại `backend/app/main.py` để phát tín hiệu Invalidate Cache tự động:

```
@app.middleware("http")
async def broadcast_live_data_changes(request, call_next):
    response = await call_next(request)
    path = request.url.path
    # Nếu là thao tác GHI thành công (POST, PUT, DELETE, status < 400)
    if request.method in {"POST", "PUT", "PATCH", "DELETE"} and response.status_code < 400:
        if "/catalog/products" in path:
            cache.delete_prefix("cat:product:")
            cache.delete_prefix("cat:search:")
        elif "/promotions" in path:
            cache.delete_prefix("promo:")
            cache.delete_prefix("voucher:")
        elif "/admin/users" in path or "/auth/roles" in path:
            cache.delete_prefix("auth:session:")
    return response
```

CHƯƠNG 5: THIẾT KẾ KỸ THUẬT & MÃ NGUỒN MINH HỌA

5.1 Thiết kế CacheManager độc lập, tuân thủ chuẩn 3 tầng

Tạo module `backend/app/core/cache.py` đóng vai trò facade, hỗ trợ hoán đổi mượt mà giữa **In-Memory (Pha 1)** và **Redis (Pha 2)** mà không làm thay đổi code tại tầng Service.

5.2 Mã nguồn hoàn chỉnh của `app/core/cache.py`

```
import time, logging
from typing import Any, Optional, Dict, Tuple

logger = logging.getLogger("cache")

class InMemoryCacheBackend:
    """Bộ nhớ đệm trong tiến trình (In-Memory), siêu nhẹ, sẵn sàng cho demo v1."""
    def __init__(self):
        self._store: Dict[str, Tuple[Any, float]] = {}

    def get(self, key: str) -> Optional[Any]:
        if key not in self._store:
            return None
        value, expire_at = self._store[key]
        if time.time() > expire_at:
            del self._store[key]
            return None
        return value

    def set(self, key: str, value: Any, ttl: int = 300) -> None:
        self._store[key] = (value, time.time() + ttl)

    def delete(self, key: str) -> None:
        self._store.pop(key, None)

    def delete_prefix(self, prefix: str) -> None:
        keys_to_del = [k for k in self._store.keys() if k.startswith(prefix)]
        for k in keys_to_del:
            self._store.pop(k, None)

class CacheService:
    """Lớp điều phối Cache toàn cục cho hệ thống Soobin Swimwear."""
    def __init__(self, backend=None):
        self.backend = backend or InMemoryCacheBackend()

    def get(self, key: str) -> Optional[Any]:
        try:
            return self.backend.get(key)
        except Exception as e:
            logger.warning(f"Cache get error for {key}: {e}")
            return None

    def set(self, key: str, value: Any, ttl: int = 300) -> None:
        try:
            self.backend.set(key, value, ttl)
        except Exception as e:
            logger.warning(f"Cache set error for {key}: {e}")

    def delete(self, key: str) -> None:
        try:
            self.backend.delete(key)
        except Exception as e:
            logger.warning(f"Cache delete error for {key}: {e}")

    def delete_prefix(self, prefix: str) -> None:
        try:
            self.backend.delete_prefix(prefix)
        except Exception as e:
            logger.warning(f"Cache delete_prefix error for {prefix}: {e}")

# Khởi tạo singleton dùng xuyên suốt ứng dụng
cache = CacheService()
```

5.3 Tích hợp thực tế vào `CatalogService`

```
from app.core.cache import cache

class CatalogService:
    def __init__(self, db: Session):
        self.db = db
        self.catalog_repo = CatalogRepository(db)

    def list_categories(self) -> List[Dict[str, Any]]:
        cache_key = "cat:categories:all"
        cached_data = cache.get(cache_key)
        if cached_data is not None:
            return cached_data

        cats = self.catalog_repo.list_categories()
        result = [{"id": c.id, "parent_id": c.parent_id, "name": c.name,
                   "slug": c.slug, "sort_order": c.sort_order, "is_active": c.is_active} for c in cats]
        cache.set(cache_key, result, ttl=86400) # Cache 24 giờ
        return result

    def get_product_detail(self, slug_or_id: str) -> Optional[Dict[str, Any]]:
        cache_key = f"cat:product:{slug_or_id}"
        cached_data = cache.get(cache_key)
        if cached_data is not None:
            return None if isinstance(cached_data, dict) and cached_data.get("__null__") else cached_data

        prod = self.catalog_repo.get_product_by_id(int(slug_or_id)) if slug_or_id.isdigit() \
               else self.catalog_repo.get_product_by_slug(slug_or_id)
        if not prod:
            cache.set(cache_key, {"__null__": True}, ttl=60) # Null Object Caching
            return None

        product_dict = self._serialize_product(prod)
        cache.set(cache_key, product_dict, ttl=1800) # Cache 30 phút
        return product_dict
```

5.4 Tích hợp Invalidation tức thì vào `AuthService` (AC-AUTH-02)

```
class AuthService:
    def validate_session(self, token: str) -> Optional[UserAccount]:
        cache_key = f"auth:session:{token}"
        user_id = cache.get(cache_key)
        if user_id:
            user = self.user_repo.get_by_id(user_id)
            if user and not user.is_locked:
                return user
            cache.delete(cache_key)
            return None

        session = self.auth_repo.get_valid_session(token)
        if not session or session.account.is_locked:
            return None
        cache.set(cache_key, session.account.id, ttl=1800)
        return session.account

    def lock_account(self, account_id: int):
        # Admin khóa tài khoản: Đáp ứng AC-AUTH-02 tức thì
        self.user_repo.update_account(account_id, {"is_locked": True})
        self.db.commit()
        cache.delete_prefix("auth:session:") # Xóa toàn bộ session của tài khoản bị khóa
```

5.5 Cấu hình Docker Compose bổ sung Redis (Pha 2)

```yaml
version: "3.8"
services:
  postgres:
    image: postgres:15-alpine
    environment:
      POSTGRES_DB: cnpm
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: password123
    volumes:
      - pgdata:/var/lib/postgresql/data

  redis:
    image: redis:7-alpine
    command: redis-server --maxmemory 128mb --maxmemory-policy allkeys-lru --save ""
    ports:
      - "6379:6379"
    restart: always

  backend:
    build: ./backend
    environment:
      - DATABASE_URL=postgresql://postgres:password123@postgres:5432/cnpm
      - REDIS_URL=redis://redis:6379/0
    depends_on:
      - postgres
      - redis
```

CHƯƠNG 6: LỘ TRÌNH TRIỂN KHAI PHÙ HỢP DEADLINE 25/11/2026

Theo mục 14 của PRD v0.2: "*Không bổ sung queue/cache/service mới trừ khi có yêu cầu cụ thể. Hoàn thành v1 ngày 25/11/2026 và demo bằng Docker Compose trên máy...*". Do đó, nhóm kỹ sư kiến trúc vạch ra lộ trình triển khai 3 giai đoạn thông minh nhằm **vừa đạt hiệu năng đỉnh cao, vừa không làm vỡ kế hoạch tiến độ MVP**:

GIAI ĐOẠN	THỜI GIAN	HẠNG MỤC CACHING TRIỂN KHAI	MỤC TIÊU & ĐẢM BẢO RỦI RO
GIAI ĐOẠN 1 (MVP Baseline)	**Hiện tại → 10/11/2026** (Tuần 1 - 2)	1. **Frontend (2 FE):** Cấu hình TanStack Query với `staleTime` và `gcTime` cho Catalog, Categories, Sizes, Colors. LocalStorage cho Guest Cart. 2. **Backend (3 BE):** Tạo `app/core/cache.py` với **In-Memory Backend**. Cache Categories, Product Details, Active Vouchers. 3. **Database:** Thêm Index cho các cột tìm kiếm và lọc.	Zero Dependency Không cần cài thêm Redis container, không làm tăng độ phức tạp của Dockerfile. Đáp ứng ngay `NFR-07` (< 3s cho 500 biến thể).
GIAI ĐOẠN 2 (Docker Demo & Tải cao)	**11/11 → 20/11/2026** (Tuần 3)	1. Tích hợp Redis vào `docker-compose.yml`. 2. Bổ sung Redis backend vào `app/core/cache.py`. 3. Kết nối Invalidation Hook vào Middleware của `app/main.py`. 4. Kiểm thử kịch bản khóa tài khoản tức thì (`AC-AUTH-02`) và đặt hàng đồng thời (`AC-CHK-01`).	Full Consistency Chuẩn hóa toàn bộ hệ thống cho buổi diễn tập tổng duyệt Docker trước hạn chót 25/11. Đảm bảo 100% test case ACID pass.
GIAI ĐOẠN 3 (Post-v1 & Mở rộng)	**Sau 25/11/2026** (Triển khai Online)	1. Cấu hình Nginx Reverse Proxy làm bộ đệm hình ảnh và Micro-caching. 2. Tích hợp Cloudflare CDN. 3. Distributed Rate Limiting chống spam đơn hàng bằng Redis Token Bucket.	Production Scale Sẵn sàng đưa sản phẩm lên môi trường Internet thực tế với hàng trăm ngàn lượt truy cập đồng thời.

Phân công nhiệm vụ cụ thể cho 5 thành viên nhóm:

- **Frontend 1 (Storefront):**
  Thiết lập TanStack Query cho trang chủ, mega menu danh mục, bộ lọc sản phẩm, chi tiết sản phẩm và Outlet. Đảm bảo khi back lại trang trước dữ liệu hiển thị tức thì từ cache trình duyệt.
- **Frontend 2 (Portal Admin & Cart):**
  Triển khai LocalStorage cho giỏ hàng khách vãng lai; xử lý làm mới dữ liệu tự động (Invalidate Queries) trong bảng quản trị sản phẩm khi Admin sửa/xóa thành công.
- **Backend 1 (Tài khoản - Catalog - Nội dung):**
  Xây dựng `app/core/cache.py`; tích hợp cache vào `CatalogService` (danh mục, chi tiết, tìm kiếm) và `AuthService` (xác thực session và cơ chế khóa user tức thì).
- **Backend 2 (Kho - Đơn hàng):**
  Đảm bảo ranh giới checkout và tồn kho TUYỆT ĐỐI dùng khóa bi-quan `with_for_update()` trong PostgreSQL; gửi tín hiệu invalidate cache sản phẩm sau khi trừ kho thành công.
- **Backend 3 (Khuyến mãi - Tích hợp - Báo cáo):**
  Tích hợp cache bảng CTKM đang chạy trong `PromotionService`; cấu hình pre-computed cache cho Dashboard thống kê doanh thu/lợi nhuận.

CHƯƠNG 7: MA TRẬN CACHING TỔNG HỢP (MASTER CACHING MATRIX)

Bảng tra cứu nhanh toàn bộ các khóa Caching, thời gian tồn tại và quy tắc vô hiệu hóa dành cho nhóm phát triển:

PHÂN HỆ	MẪU KHÓA (KEY PATTERN)	VỊ TRÍ LƯU	TTL	ĐIỀU KIỆN XÓA (INVALIDATION TRIGGER)	TIÊU CHUẨN NGHIỆM THU
Danh mục & Master	`cat:categories` `cat:sizes` `cat:colors`	React Query + In-Memory/Redis	24 giờ	Admin sửa danh mục hoặc cấu hình size/màu qua portal.	NFR-07 (Tải nhanh dưới 3s)
Sản phẩm Chi tiết	`cat:product:{id_or_slug}`	In-Memory / Redis	30 phút	1. Cập nhật SP (`PUT /products/{id}`) 2. Xóa mềm SP (`DELETE`) 3. Tạo đơn trừ kho thành công.	AC-CAT-01, AC-CAT-04 (Snapshot giá cũ không đổi)
Bộ lọc Tìm kiếm	`cat:search:{hash_query}`	In-Memory / Redis	5 phút	Khi có bất kỳ thay đổi nào trong bảng `production_product` (xóa tiền tố `cat:search:*`).	AC-CAT-02 (Không lặp hoặc sót kết quả phân trang)
Sản phẩm Outlet	`cat:outlet:list`	In-Memory / Redis	10 phút	Khi Admin thêm/sửa CTKM giảm giá sản phẩm.	Mục 5 PRD (Giá sale là giá sau KM)
CTKM Đang chạy	`promo:active_list`	In-Memory / Redis	15 phút	Admin tạo/sửa/tắt chương trình khuyến mãi.	BR-PROMO-01-11 (So sánh 3 phương án KM)
Tra cứu Voucher	`voucher:info:{code}`	In-Memory / Redis	5 phút	1. Tạo đơn dùng voucher thành công (trừ lượt) 2. Hủy đơn hoàn lượt voucher.	AC-CHK-03, AC-INV-06 (Không trừ hoặc hoàn lặp 2 lần)
Phiên làm việc (Session RBAC)	`auth:session:{token}`	In-Memory / Redis	30 phút	1. Đăng xuất 2. **Admin khóa user** 3. **Admin sửa permission của role**.	**AC-AUTH-02** (Khóa/đổi quyền có hiệu lực với request kế tiếp)
Giỏ hàng Khách	`soobin_guest_cart`	Browser LocalStorage	Vĩnh viễn (Client)	1. Khách xóa món 2. Đăng nhập và gộp giỏ thành công vào DB (Merge Cart).	BR-PRD-CART-01, BR-PRD-CART-02
Dashboard Thống kê	`dashboard:overview_stats`	In-Memory / Redis	15 phút	Đơn hàng chuyển trạng thái `Đã giao` (xuất hóa đơn) hoặc `Đã hoàn tiền` .	Mục 11 PRD (Doanh thu, lợi nhuận snapshot)

CHƯƠNG 8: KỊCH BẢN KIỂM THỬ & NGHIỆM THU CACHING

8.1 Các kịch bản kiểm thử trọng tâm (Test Cases)

Nhóm QA/Tester cần thực thi 5 ca kiểm thử chuyên sâu nhằm đảm bảo Caching không vi phạm quy tắc nghiệp vụ:

MÃ TEST	KỊCH BẢN KIỂM THỬ	CÁC BƯỚC THỰC HIỆN	KẾT QUẢ KỲ VỌNG
TC-CACHE-01	Kiểm tra Tốc độ Tải Danh mục & Chi tiết (NFR-07)	1. Tạo 100 sản phẩm với 500 biến thể. 2. Dùng k6 / Apache Bench gửi 1000 requests vào `/api/v1/catalog/products`. 3. So sánh thời gian phản hồi trước và sau khi bật cache.	• Trước Caching: 800ms - 2200ms. • Sau Caching (Cache Hit): **< 15ms**. • Tải trang hoàn tất < 1.5 giây.
TC-CACHE-02	Xác minh Khóa Tài khoản Tức thì (AC-AUTH-02)	1. User A đăng nhập, lấy session token. 2. User A gọi API `/api/v1/cart` (Cache hit: 200). 3. Admin gọi API khóa tài khoản User A. 4. User A lập tức gọi lại API `/api/v1/cart`.	Ngay tại bước 4, hệ thống phải trả về lỗi **401 Unauthorized** ("Tài khoản đã bị khóa") ngay lập tức, không cho phép tiếp tục thao tác dù token chưa hết hạn.
TC-CACHE-03	Cạnh tranh Hàng Cuối & Tính Nhất quán Kho (AC-CHK-01 & E2E-06)	1. Biến thể áo tắm X chỉ còn tồn kho = 1. 2. Trang chi tiết của 2 khách hiển thị Còn hàng (từ cache). 3. Khách A và Khách B cùng bấm Đặt hàng tại cùng 1 thời điểm (script song song 2 request).	**Chỉ đúng 1 đơn hàng được tạo thành công**; đơn thứ 2 báo lỗi hết hàng. Tồn kho trong PostgreSQL về 0, không âm. Cache chi tiết sản phẩm được xóa.
TC-CACHE-04	Tính Toàn vẹn Vô hiệu hóa khi Sửa Sản phẩm (AC-CAT-04)	1. Xem chi tiết sản phẩm `/products/ao-tam-01` (giá 300.000đ). 2. Quản lý sửa giá thành 350.000đ. 3. Khách F5 lại trang sản phẩm.	Trang sản phẩm hiển thị ngay giá mới 350.000đ (do cache cũ đã bị xóa). Các đơn hàng cũ đã tạo vẫn giữ nguyên giá snapshot 300.000đ.
TC-CACHE-05	Chống Xuyên thủng Cache (Cache Penetration)	Gửi liên tục 100 requests vào `/products/invalid-slug-999`.	Request đầu tiên vào DB (trả về 404). 99 request sau đó được chặn trả về 404 trực tiếp từ Null Cache, không sinh query PostgreSQL.

8.2 Giám sát & Chỉ số Đo lường Hiệu quả (KPIs)

- **Tỉ lệ Cache Hit (Cache Hit Ratio):** Mục tiêu đạt **≥ 85%** cho các API Storefront công khai (Catalog, Categories, Outlet).
- **Thời gian phản hồi P95 (95th Percentile Latency):** Dưới **50ms** đối với các request Cache Hit.
- **Số lượng Database Connections:** Giảm tải từ đỉnh 30 kết nối xuống duy trì ổn định dưới 5 - 8 kết nối PostgreSQL ngay cả trong các bài test chịu tải nặng.